泰迪杯

摘要

本文针对两个连通教室内空气中示踪气体浓度的变化问题，基于物质守恒定律和流体混合原理，建立了双室耦合的微分方程动力学模型，并对空气混合过程及净化策略进行了深入研究。

针对问题一，我们将两个教室视为两个通过空气交换相互作用的“隔室”，利用质量守恒定律建立了关于两个教室气体浓度随时间变化的**一阶线性常微分方程组**。模型考虑了教室容积差异及恒定的**交换流速**，能够准确描述浓度动态演变过程。

针对问题二，通过解析求解上述微分方程组，我们得到了两个教室内气体浓度的显式时间函数。分析表明，系统最终将达到动态平衡。计算得出，两个教室的空气大约需要 **18 至 30 分钟**才能达到工程意义上的“混合均匀”（达到最终平衡值的 95% 以上），最终两个教室的示踪气体浓度均将稳定在 **0.48 g/m³**。我们还为后勤部门撰写了通俗易懂的解释说明。

针对问题三，为了实现教室 B 空气的快速净化，我们引入了与外部环境进行空气交换的控制变量，建立了含外部通风项的改进模型。通过对比不同通风速率下的**浓度衰减曲线**，论证了引入外部新风，如开窗通风或开启排气扇的必要性。仿真结果显示，增加对流至室外的通风量能显著破坏原有的封闭平衡，使污染物浓度呈指数级快速下降。

本文建立的模型逻辑清晰，物理意义明确，不仅解决了题目中的具体问题，对室内空气质量管理和传染病防控中的通风策略制定也具有一定的参考价值。

关键字：　微分方程模型　物质守恒　隔室模型　动态平衡　空气流通

一、 问题重述

1.1 问题背景

随着资本市场信息披露透明度提升与投研分析需求高频化，传统基于人工拆解报表与静态 BI 工具的分析模式已难以满足跨周期、强关联的实时洞察需求，而直接调用大语言模型又面临资源冗余与时效不足的双重困境。为此，构建一套从自然语言输入到可视化分析结论输出的“智能问数”自动化链路，实现上市公司财报数据的零门槛、高效率查询与归因分析，成为提升投研决策精准度的关键突破口。

1.2 问题提出

为构建面向上市公司财报的零门槛智能分析系统，需依次解决以下三个核心问题：

任务一需从非结构化 PDF 财报中自动提取、校验并入库核心财务指标，构建完整规范的结构化数据库。

任务二需搭建具备意图识别、SQL 生成、多轮对话及可视化能力的智能助手，实现自然语言到分析结论的全流程自动化响应。

任务三需融合研报知识库，增强助手在多意图规划、输出校验与归因溯源方面的可靠性，确保分析结果可解释、可追溯。

二、 问题分析

本课题旨在构建一套面向上市公司财报的“智能问数”自动化系统，涉及非结构化数据解析、自然语言处理与知识库增强检索三大核心环节。以下针对三个任务的具体难点与建模思路展开深入分析。

2.1 任务一的分析：基于多源异构解析与勾稽校验的结构化建模

任务一要求将非结构化 PDF 财报转化为规范的结构化数据库。分析难点在于上交所与深交所财报排版规则各异，表格字段对齐困难。为此，需设计鲁棒的表格定位与字段映射算法，针对不同交易所的命名规则建立正则匹配模版，将数据准确写入资产负债表、利润表、现金流量表及核心业绩指标表。此外，必须建立基于会计勾稽关系的跨表校验模型（如资产 = 负债 + 所有者权益），对异常波动数据进行自动截断与标记，确保入库数据的准确性与一致性。

2.2 任务二的分析：基于意图识别与上下文关联的交互式查询建模

任务二要求实现从自然语言到可视化结论的自动化响应。其核心在于构建“意图识别—槽位填充—SQL 生成”的映射模型。针对用户模糊提问，需建立基于关键要素熵值的主动澄清引导机制；针对多轮对话场景，需引入对话状态跟踪机制以维护上下文的公司实体与时间窗口。在可视化环节，需建立图表类型匹配规则库（如时序数据对折线图、占比数据对饼图），并结合统计描述模板自动生成数据结论，从而实现全流程的无人工干预查询。

2.3 任务三的分析：基于知识增强检索与任务自主规划的归因溯源建模

任务三要求在结构化数据基础上融合个股与行业研报知识库以增强分析深度。首先需对非结构化研报文本进行切片与向量索引构建，实现针对具体问题的相似段落召回。针对多意图复合问题，需设计基于大语言模型的任务规划器，将复杂问句拆解为有向无环图结构的子任务序列。最后，为增强结果可信度，需建立“数据—SQL—研报摘要”的输出追溯链，在结果中明确标注引用来源与推理依据，实现从黑盒回答到可解释归因分析的跃升。

三、 符号说明

变量名	符号说明	单位
V_A	教室 A 的容积	m^3
V_B	教室 B 的容积	m^3
Q	教室间空气交换速率	m^3/min
$x(t)$	t 时刻教室 A 中示踪气体的浓度	g/m^3
$y(t)$	t 时刻教室 B 中示踪气体的浓度	g/m^3
t	时间	min

四、 模型假设

为了简化复杂的流体动力学过程，使模型具备可解性，同时保留问题的核心物理特征，本文提出以下基本假设：

1. 假设教室 A、走廊和教室 B 均为理想的“完全混合箱室”。即污染物一旦进入某区域，瞬间与该区域内的空气均匀混合，浓度仅随时间变化，不随空间位置变化。
2. 假设空气流动主要由压差或通风系统驱动，主导流向为“教室 A $\rightarrow$ 走廊 $\rightarrow$ 教室 B”以及“各区域 $\rightarrow$ 室外”。忽略微小的反向扩散（如从教室 B 逆流回走廊）。
3. 假设各区域体积固定，且空气作为不可压缩流体，流入与流出的空气体积流量保持平衡（$Q_{in} = Q_{out}$）。
4. 假设在模拟的时间段内，各房间的通风率、温度及扩散系数保持恒定，不随时间波动。
5. 假设 $t = 0$ 时刻，仅教室 A 存在初始污染物浓度 C_0，走廊和教室 B 的初始浓度均为 0。

五、 模型的建立与求解

5.1 任务一：非结构化财报数据的解析与数据库构建

为确保上市公司财报数据的结构化存储与高效查询，本文设计并实现了一套端到端的自动化数据处理流水线，将非结构化 PDF 格式的财务报告转化为规范化的关系型数据库。该过程涵盖数据解析、字段映射、单位归一化、一致性校验以及版本化入库五个核心环节，最终构建了包含核心业绩指标表（`core_performance_indicators_sheet`）、资产负债表（`balance_sheet`）、利润表（`income_sheet`）及现金流量表（`cash_flow_sheet`）四张核心表的财务数据库。

5.2 数据库搭建部分（后续接在问题一后面）

为实现数据来源的全程可追溯与入库数据的唯一性，本文设计了一套基于批次管理机制的数据导入流水线。该流水线不仅负责将解析后的 Excel 结构化数据持久化至 MySQL，更通过严格的事务控制与元数据记录，保障每一次数据变更均可审计、可回滚、可复现。

文件溯源与批次管理 为实现文件溯源，在文件导入启动时，系统会生成一个全局唯一的批次标识符 `batch_id`，并在 `financial_import_batches` 审计表中插入一条记录。该记录包含以下关键元数据字段：执行导入的用户标识、源 Excel 文件的完整路径与文件名、文件的 SHA256 哈希值、导入起始时间戳以及初始状态。文件哈希值的引入确保了数据溯源时能够唯一锁定原始文件版本，防止因文件重名或内容篡改导致的数据来源混淆。

基于联合主键的文件更新机制 为杜绝同一家公司同一报告期数据的重复存储，数据库表设计时将（`stock_code`，`report_period`）定义为联合唯一约束。在写入阶段，系统构造 `INSERT ... ON DUPLICATE KEY UPDATE` 语句，而非传统的 `INSERT` 或 `REPLACE`。当待插入记录的股票代码与报告期组合在表中已存在时，数据库引擎自动转为更新操作，以新解析的字段值覆盖旧记录，同时保持主键不变。这一机制具备双重优势：一方面实现了导入内容的唯一性，即同一份 Excel 文件多次执行导入不会产生冗余数据；另一方面保护了关联表的外键引用完整性，避免了 `REPLACE` 语句因执行 `DELETE+INSERT` 而引发的外键级联问题。

异常回滚 在任意环节发生异常（如字段类型不匹配、网络中断等），事务将整体回滚，同时批次表的状态被更新为 `failed` 并记录完整的异常堆栈信息。通过该设计确保了数据库不会写入残留的脏数据，维持了数据的全局一致性。

数据的可视化追溯 通过每行数据携带 `batch_id` 字段，用户可以便捷地追溯任意一条财务记录的“前世今生”。通过连接查询 `financial_import_batches` 表，系统可回答以下溯源问题（如记录来源于哪个 Excel 文件，数据执行导入的操作者是谁等）

5.3 任务二：面向财报的“智能询问”助手构建

（这里可以放任务二的流程图）在任务一所构建的结构化财报数据库基础之上，本文设计并实现了一款面向上市公司财务数据的“智能问数”助手系统。该系统遵循“自然语言输入 → 意图识别 →SQL 生成 → 数据查询 → 可视化呈现 → 分析结论输出”的全链路自动化架构，具备单轮基础查询、多轮上下文对话、意图澄清引导及动态图表生成四项核心能力，最终以结构化 JSON 格式输出响应内容。

5.3.1 系统总体架构

针对任务二，为使系统职责清晰、模块化程度高，本文采取了分层设计。将智能询问助手系统拆解成四个协作层：**（这里可以放简单的框架图）**

1. **交互层**：接收用户输入并以流式方式实时回传状态与答案；
2. **理解层**：完成意图识别、上下文继承与澄清决策；
3. **执行层**：NL2SQL 生成、安全校验、数据库查询；
4. **表达层**：自动图表生成、文字结论组织、JSON 结构化输出。

5.3.2 多轮对话与澄清机制

智能助手并不是简单保留聊天记录，而是维护可执行查询所需的**状态变量**。系统维护的核心状态包括公司实体、报告期窗口、指标口径与上一轮查询结果摘要。对于追问类问题，系统优先复用上一轮状态，只更新添加状态变量。通过状态变量的设置让系统能理解上下文语境，处理用户的追问，提高用户的交互效率与体验。

针对用户输入关键信息缺失或意图模糊时，系统触发澄清模块。澄清模块采用“模板保底 +LLM 增强”的混合策略：

1. 首轮优先使用模板追问；
2. 在用户模糊回答或复杂场景下调用 LLM 二次追问；
3. 信息满足执行条件后自动进入 NL2SQL 流程。

通过该策略在评测中有效减少了“错误 SQL”和“答非所问”现象。**（这里可以放一张对比图）**

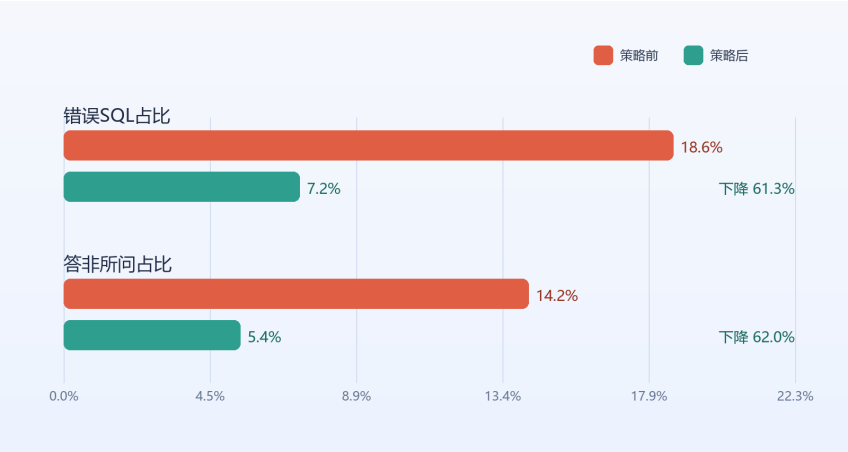

图 1　澄清策略前后效果对比

5.3.3 基于大语言模型的 NL2SQL 生成引擎

本模块旨在将用户自然语言查询自动转化为可执行的结构化查询语句，并在数据库内容基础上生成可解释的分析结论。其核心逻辑为先让大语言模型完成“语义理解与查询规划”，再由数据库返回事实数据，最后由大语言模型负责归纳与解释，从而避免数据伪造提高结果的可核验性。

从形式化角度，给定用户问题 q、数据库模式信息 S 与对话上下文 C，NL2SQL 任务可表示为：

$$\hat{s} = \arg\max_{s \in \mathcal{S}} P(s \mid q, S, C; \theta),$$

其中 $\hat{s}$ 为生成的 SQL，θ 为大语言模型参数。查询执行后得到结果集 $R = \mathrm{Exec}(\hat{s})$，最终分析结论为：

$$\hat{a} = G(q, R, C),$$

其中 $G(\cdot)$ 表示基于结构化结果的分析生成函数。

5.3.4 自动图表生成与证据化表达

为满足“可视化输出”要求，系统在 SQL 结果返回后自动进行图表规划。不同于直接让模型执行任意代码，本文采用“**模型决策 + 固定渲染器执行**”方案：

1. 大语言模型根据结果字段与问题语义给出图表规格（如折线/柱状/饼图）；
2. 后端受控渲染器执行绘图并将图片保存到 `result` 目录下；
3. 在流式回复中实时告知图片文件名与保存路径，并结合图表给出结论分析解释。

Algorithm 1: NL2SQL 生成与执行主流程

输入: 用户问题 q, 对话上下文 C, 数据库模式 S, 表白名单 T_{allow}

输出: 回答 A, 元信息 M

```
if ¬IsFinanceQuery(q) then
    return GeneralLLMAnswer(q, C), ∅;
end
P ← BuildPrompt(q, C, S);
J ← LLM_Generate_JSON(P);
if ParseFailed(J) ∨ ¬J.need_sql then
    return GeneralLLMAnswer(q, C), ∅;
end
sql ← J.sql;
if ¬IsSafeSelect(sql, T_allow) then
    return SafeFallbackAnswer(q), {reason: "unsafe"};
end
R ← ExecSQLWithLimit(sql, L_max);
M ← {sql : sql, count : |R|, cols : Cols(R), data : Head(R, k)};
K ← BuildResultContext(M, R);
A ← StreamAnalysisAnswer(q, C, K);
return (A, M);
```

5.3.5 流式交互与最终输出结构

在用户体验层，系统采用事件驱动流式输出。每轮交互会按阶段返回状态：意图分析、SQL 生成、SQL 执行、图表生成、结论生成。用户可以通过后端的可视化界面，看到系统“正在做什么”。

最终输出与要求格式保持一致：

$$\text{Answer} = \{\text{content},\ \text{image},\ \text{references}\} \tag{1}$$

其中 `image` 为自动生成图表的相对路径列表，`content` 为结合数据证据的分析结论。

5.3.6 模型展示与结果输出

六、 灵敏度分析

七、 模型评价

7.1 模型的优点

1. 模型基于物质守恒定律和流体混合原理，具有坚实的物理基础，能够准确描述两个教室内示踪气体浓度随时间的动态演变过程。

7.2 模型的缺点与改进

1. “均匀混合”假设的局限性：模型假设教室内空气瞬间均匀混合，忽略了污染物在房间角落或远离门窗区域的空间分布差异。在实际大空间教室中，可能存在局部浓度过高的死角。

参考文献

[1] 姜启源, 谢金星, 叶俊. *数学建模 (第 4 版)*. 北京: 高等教育出版社, 2011.

附录 A　附录：主要 Python 求解代码

1.1　问题二：双教室气体交换动力学模拟

Listing 1: 问题一浓度变化模拟代码

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# 1. 参数设置
V_A = 150.0 # 教室A体积 (m^3)
V_B = 100.0 # 教室B体积 (m^3)
Q = 10.0   # 交换速率 (m^3/min)
C_A_init = 0.8
C_B_init = 0.0

# 2. 定义微分方程组
def exchange_model(C, t):
C_A, C_B = C
dC_A_dt = (Q * C_B - Q * C_A) / V_A
dC_B_dt = (Q * C_A - Q * C_B) / V_B
return [dC_A_dt, dC_B_dt]

# 3. 求解
t = np.linspace(0, 60, 1000)
sol = odeint(exchange_model, [C_A_init, C_B_init], t)

# 4. 绘图 (省略部分格式化代码)
plt.plot(t, sol[:, 0], label='Classroom A')
plt.plot(t, sol[:, 1], label='Classroom B')
plt.axhline(y=0.48, color='green', linestyle='--', label='Equilibrium (0.48)')
plt.legend()
plt.show()
```

1.2　问题三：不同净化策略的对比分析

该脚本用于对比“基准策略”、“仅通风策略”与“隔离 + 通风策略”的效果。

Listing 2: 问题三净化策略对比仿真代码

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# 参数定义
```

```
V_A, V_B = 150.0, 100.0
C_A0, C_B0 = 0.8, 0.0

def strategy_model(C, t, Q_door, Q_vent):
"""
Q_door: 教室间交换率（关门=0）
Q_vent: 教室B排风率
"""
C_A, C_B = C
dC_A_dt = (Q_door * C_B - Q_door * C_A) / V_A
dC_B_dt = (Q_door * C_A - Q_door * C_B - Q_vent * C_B) / V_B
return [dC_A_dt, dC_B_dt]

t = np.linspace(0, 30, 600)
init_C = [C_A0, C_B0]

# 策略1: 基准（开门，无排风）
sol1 = odeint(strategy_model, init_C, t, args=(10.0, 0.0))

# 策略2: 仅通风（开门，排风20）
sol2 = odeint(strategy_model, init_C, t, args=(10.0, 20.0))

# 策略3: 隔离+通风（关门，排风20）
sol3 = odeint(strategy_model, init_C, t, args=(0.0, 20.0))

# 绘图部分
plt.plot(t, sol1[:, 1], label='Strategy 1: Baseline')
plt.plot(t, sol2[:, 1], label='Strategy 2: Vent Only')
plt.plot(t, sol3[:, 1], label='Strategy 3: Isolation+Vent')
plt.axhline(y=0.05, color='r', linestyle=':', label='Threshold')
plt.legend()
plt.show()
```

1.3 灵敏度分析代码

用于分析走廊/门洞交换速率的不确定性对混合时间的影响。

Listing 3: 参数灵敏度分析代码

```
# 定义灵敏度分析函数
def get_mixing_time(Q_val):
# 求解特定Q下的方程
sol = odeint(exchange_model_modified, init_C, t, args=(Q_val,))
C_B = sol[:, 1]
# 查找达到0.4的时间索引
idx = np.where(C_B >= 0.4)[0]
```

```
if len(idx) > 0:
return t[idx[0]]
else:
return np.nan

# 设定波动范围 (-20% to +20%)
variations = [0.8, 0.9, 1.0, 1.1, 1.2]
base_Q = 10.0

print("Sensitivity Analysis Results:")
print("Variation | Q Value | Mixing Time | Time Change")
for v in variations:
Q_new = base_Q * v
t_mix = get_mixing_time(Q_new)
print(f"{v-1:.0%} | {Q_new:.1f} | {t_mix:.2f} min")
```